PYTHON

The Complete Reference: Zero to DSA

An exhaustive, beginner-to-advanced study guide

covering every core sub-topic, with theory, full method references,

line-by-line code examples, and visual diagrams

✓ Part I — Python Fundamentals (every data type, in full depth)

✓ Part II — Functions, OOP, Errors, Files, Modules, Iterators

✓ Part III — Data Structures & Algorithms (complete)

✓ Part IV — Patterns, Advanced Topics & Practice Roadmap

Table of Contents

PART IV — PRACTICE ROADMAP

PART I

Python Fundamentals — every core data type explained completely

1. Introduction & Setting Up Python

Python is a high-level, interpreted, dynamically-typed programming language. 'Interpreted' means code runs line-by-line through an interpreter (CPython, the standard implementation) rather than being compiled to machine code ahead of time. This makes Python slower than C/C++ for raw computation, but far faster to write, read, and debug — which is why it's the most common first language and the language of choice for data science, scripting, and technical interviews.

Why learn Python first?

- Readable syntax close to plain English — you focus on logic, not punctuation.
- Huge standard library ('batteries included') and third-party ecosystem (pip).
- Used heavily in interviews and DSA practice (LeetCode, HackerRank, GeeksforGeeks).
- Easiest language to translate flowchart logic directly into working code.

Installing Python & Running Code

Download from **python.org**, and on the installer check "Add Python to PATH". Verify with:

```
python --version
# or
python3 --version
```

- Interactive mode (REPL) — type 'python' in a terminal to get a >>> prompt for testing snippets instantly.
- Script mode — save code in a .py file and run with 'python filename.py'.
- IDEs/editors — VS Code, PyCharm, Jupyter Notebook are the most common.

Your First Program & Comments

```
print("Hello, World!")

# This is a single-line comment
"""
This is a multi-line string,
often used as a comment block or docstring.
"""
```

- print(...) is a built-in function that displays output on the screen.
- # starts a single-line comment — ignored by the interpreter.
- Triple-quoted strings (''' or """) spanning multiple lines are often used as comments or docstrings.

> **NOTE:** Python uses indentation (spaces, conventionally 4) instead of curly braces {} to define code blocks. Mixing tabs and spaces causes errors — pick one and stay consistent.

2. Variables, Data Types, Type Casting & Operators

Variables & Naming Rules

A variable is a named reference to an object in memory. Python is dynamically typed — types are inferred from values, and a variable can be reassigned to a different type entirely.

```
age = 25
age = "twenty five" # legal - age now refers to a string object

# Naming rules:
# - must start with a letter or underscore, not a digit
# - can contain letters, digits, underscores
# - case-sensitive (age != Age)
# - cannot be a reserved keyword (if, for, class, etc.)

# Multiple assignment
x, y, z = 1, 2, 3
a = b = c = 0 # all three point to the same object 0

# Constants (by convention only, Python has no true constants)
MAX_SIZE = 100 # ALL_CAPS signals "do not change this"
```

Type Checking & Type Casting

```
x = 10
print(type(x)) # <class 'int'>
print(isinstance(x, int)) # True

# Explicit type casting
print(int("42")) # 42 str -> int
print(float("3.14")) # 3.14 str -> float
print(str(99)) # "99" int -> str
print(bool(0)) # False 0, "", [], {}, None are all "falsy"
print(bool(5)) # True any non-zero/non-empty value is "truthy"
print(list("abc")) # ['a','b','c']
print(tuple([1,2,3])) # (1,2,3)
```

Core Data Types Overview

Type	Example	Mutable?	Description
int	10, -5	No	Whole numbers, unlimited precision
float	3.14	No	Decimal / floating-point numbers
complex	2+3j	No	Numbers with a real and imaginary part

str	'hi'	No	Sequence of characters
bool	True/False	No	Logical values (subclass of int)
list	[1,2,3]	Yes	Ordered, indexable collection
tuple	(1,2,3)	No	Ordered, immutable collection
set	{1,2,3}	Yes	Unordered, unique elements
frozenset	frozenset({1,2})	No	Immutable version of a set
dict	{'a':1}	Yes	Key -> value mapping
NoneType	None	—	Represents the absence of a value

Operators — Every Category

Arithmetic

```
a, b = 7, 2
a + b # 9 addition
a - b # 5 subtraction
a * b # 14 multiplication
a / b # 3.5 true division (always returns float)
a // b # 3 floor division (rounds toward negative infinity)
a % b # 1 modulus (remainder)
a ** b # 49 exponentiation
```

Comparison

```
a == b # equal to
a != b # not equal to
a > b, a < b, a >= b, a <= b # standard comparisons
```

Logical

```
a > 1 and b > 1 # True only if BOTH are true
a > 1 or b > 10 # True if AT LEAST ONE is true
not (a > b) # inverts a boolean
```

Assignment & Augmented Assignment

```
x = 5
x += 3 # x = x + 3 -> 8
x -= 2 # x = x - 2 -> 6
x *= 4 # x = x * 4 -> 24
x //= 5 # x = x // 5 -> 4
x **= 2 # x = x ** 2 -> 16
x %= 5 # x = x % 5 -> 1
```

Identity vs Equality (is vs ==)

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b) # True - same VALUES
print(a is b) # False - different OBJECTS in memory
c = a
print(a is c) # True - c points to the SAME object as a
```

NOTE: Use '==' to compare values, and 'is' only to check identity (e.g., 'x is None'). Confusing the two is a very common beginner bug.

Membership Operators

```
nums = [1, 2, 3]
print(2 in nums) # True
print(5 not in nums) # True
```

Bitwise Operators

```
a, b = 6, 3 # binary: 110 and 011
a & b # 2 AND - bits set in both
a | b # 7 OR - bits set in either
a ^ b # 5 XOR - bits set in exactly one
~a # -7 NOT - inverts all bits
a << 1 # 12 left shift - multiply by 2
a >> 1 # 3 right shift - divide by 2
```

Operator Precedence (high to low, simplified)

- () parentheses — always evaluated first
- ** exponentiation
- +x, -x, ~x unary operators
- *, /, //, % multiplication/division group
- +, - addition/subtraction
- <<, >> bitwise shifts
- &, ^, | bitwise AND/XOR/OR
- comparisons (==, !=, <, >, <=, >=, is, in)
- not, then and, then or (logical operators, lowest precedence)

3. Numbers (int, float, complex) & the math/random modules

Integers & Floats

Python integers have unlimited precision (they grow as large as memory allows). Floats follow the IEEE-754 double-precision standard, which means some decimal values cannot be represented exactly.

```
print(2 ** 100) # huge integer, computed exactly
print(0.1 + 0.2) # 0.30000000000000004 -> floating point rounding
print(round(0.1 + 0.2, 2)) # 0.3 -> round() fixes display precision
print(divmod(17, 5)) # (3, 2) -> (quotient, remainder) in one call
print(abs(-9)) # 9
print(pow(2, 10)) # 1024 same as 2 ** 10
print(pow(2, 10, 1000)) # 24 -> (2**10) % 1000, fast modular exponentiation
```

The math module

Method / Syntax	What it does
`math.sqrt(x)`	Square root of x
`math.floor(x) / math.ceil(x)`	Round down / up to nearest integer
`math.factorial(n)`	n! (factorial)
`math.gcd(a, b)`	Greatest common divisor
`math.pow(x, y)`	x raised to y, always returns a float
`math.log(x, base)`	Logarithm of x to the given base
`math.pi, math.e`	Mathematical constants
`math.inf, math.nan`	Infinity and 'not a number'

The random module

Method / Syntax	What it does
`random.random()`	Random float between 0.0 and 1.0
`random.randint(a, b)`	Random integer N such that a <= N <= b
`random.choice(seq)`	Random element from a sequence
`random.shuffle(list)`	Shuffles a list in place
`random.sample(seq, k)`	k unique random elements from a sequence

NOTE: Complex numbers (e.g., 2+3j) are rarely needed outside scientific computing, but Python supports them natively: z = 2+3j; print(z.real, z.imag).

4. Strings — In Full Depth

A string is an ordered, **immutable** sequence of Unicode characters. Immutable means any 'modifying' operation actually builds and returns a brand-new string.

Creation, Indexing & Slicing

```
s = "Hello, Python!"
s2 = 'single quotes work too'
s3 = """multi-line
string"""
raw = r"C:\new_folder" # raw string - backslashes are literal

print(s[0]) # H -> indexing starts at 0
print(s[-1]) # ! -> negative index counts from the end
print(s[0:5]) # Hello -> slice [start:stop), stop excluded
print(s[7:]) # Python! -> from index 7 to the end
print(s[:5]) # Hello -> from start to index 5
print(s[::2]) # Hlo yhn! -> every 2nd character
print(s[::-1]) # !nohtyP ,olleH -> reversed string
```

Escape Sequences

Sequence	Meaning
\n	Newline
\t	Tab
\\	Backslash
\' and \"	Quote characters
\r	Carriage return

String Formatting — 3 Ways

```
name, score = "Asha", 91.5

# 1) f-strings (Python 3.6+, recommended)
print(f"{name} scored {score:.1f}%")

# 2) .format()
print("{} scored {:.1f}%".format(name, score))

# 3) %-formatting (legacy/older style)
print("%s scored %.1f%%" % (name, score))
```

Complete String Methods Reference

Method / Syntax	What it does

`s.upper() / s.lower()`	Convert to UPPERCASE / lowercase
`s.title() / s.capitalize()`	Title Case Each Word / Capitalize First Letter
`s.swapcase()`	Swap upper and lower case
`s.strip() / lstrip() / rstrip()`	Remove whitespace (or given chars) from both/left/right
`s.split(sep) / rsplit(sep)`	Split into a list of substrings
`s.splitlines()`	Split a multi-line string into a list of lines
`sep.join(list)`	Join a list of strings using sep as glue
`s.replace(old, new)`	Replace all occurrences of a substring
`s.find(sub) / rfind(sub)`	Index of first/last occurrence, or -1 if not found
`s.index(sub) / rindex(sub)`	Like find(), but raises ValueError if not found
`s.count(sub)`	Count non-overlapping occurrences of a substring
`s.startswith(x) / endswith(x)`	Check prefix / suffix
`s.isalpha() / isdigit() / isalnum()`	Check if all chars are letters / digits / alphanumeric
`s.isspace() / isupper() / islower()`	Check whitespace-only / all-upper / all-lower
`s.zfill(width)`	Pad with leading zeros to reach a width
`s.center(w) / ljust(w) / rjust(w)`	Pad string to width w, centered/left/right
`s.encode('utf-8')`	Convert string to bytes
`len(s)`	Number of characters

NOTE: Strings are sequences, so they support 'in' (substring check), len(), iteration with 'for ch in s', and concatenation with '+'. But '+' in a loop is O(n) each time — prefer ''.join(list_of_strings) when building a string from many pieces.

5. Control Flow: Conditionals, match-case & Loops

if / elif / else

Conditional statements let your program make decisions and execute different code paths depending on whether a condition is True or False. Only one branch ever runs.

```
age = 20
if age < 13:
    print("Child")
elif age < 20:
    print("Teenager")
else:
    print("Adult")
# Output: Adult
```

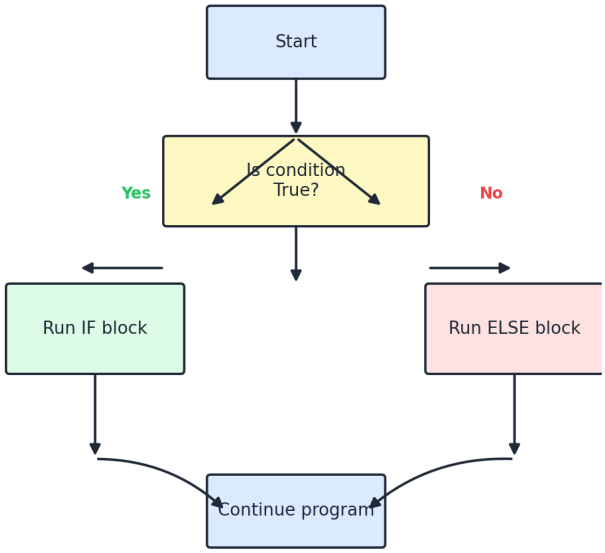

Figure 5.1 — Flow of an if / else decision

Ternary (Conditional) Expression

```
age = 20
status = "Adult" if age >= 18 else "Minor"
print(status) # Adult
```

match-case (Structural Pattern Matching, Python 3.10+)

match-case is Python's version of a switch statement, and can also match on structure (not just exact values).

```
def handle(command):
    match command:
        case "start":
            return "Starting..."
        case "stop":
            return "Stopping..."
        case ["move", direction]: # matches a 2-item list pattern
            return f"Moving {direction}"
        case _: # default / wildcard case
            return "Unknown command"

print(handle("start")) # Starting...
print(handle(["move", "left"])) # Moving left
```

for loops & range()

```
for i in range(5): # 0,1,2,3,4 (stop excluded)
    print(i)
for i in range(2, 10, 2): # start, stop, step -> 2,4,6,8
    print(i)

fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)

for i, fruit in enumerate(fruits): # index + value together
    print(i, fruit)
```

while loops

```
count = 0
while count < 3:
    print("count is", count)
    count += 1 # without this, the loop never ends!
```

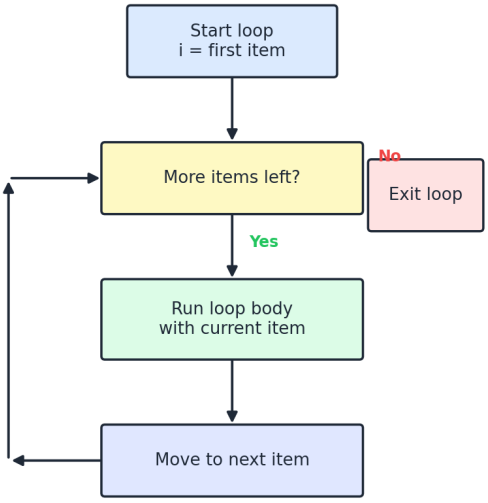

Figure 5.2 — General loop execution flow

NOTE: An infinite loop happens when the while condition never becomes False. Always ensure something inside the loop moves it toward termination.

break, continue, pass, and the loop 'else' clause

- break — immediately exits the loop entirely.
- continue — skips the rest of the current iteration, moves to the next.
- pass — does nothing; a placeholder where a statement is syntactically required.
- else (on a loop) — runs only if the loop completed WITHOUT hitting a break.

```
for i in range(10):
    if i == 5:
        break
    if i % 2 == 0:
        continue
    print(i) # prints 1, 3

for n in range(2, 10):
    for x in range(2, n):
        if n % x == 0:
            break
    else:
        print(n, "is prime") # runs since no break occurred (no divisor
found)
```

Nested Loops

```
for i in range(3):
    for j in range(3):
        print(i, j)
# Outer loop runs 3 times; for EACH outer iteration, inner loop runs fully (3
times)
# Total iterations = 3 * 3 = 9 -> this nested pattern is the basis of O(n^2)
algorithms
```

6. Lists — In Full Depth

A list is an ordered, **mutable**, indexable collection that can hold mixed data types. Internally, a Python list is a dynamic array: a resizable, contiguous block of memory holding references to objects, which is why index access is O(1).

Creation, Indexing & Slicing

```
nums = [10, 20, 30, 40, 50]
mixed = [1, "two", 3.0, [4, 5]] # mixed types, even nested
empty = []
from_range = list(range(5)) # [0, 1, 2, 3, 4]

print(nums[0]) # 10
print(nums[-1]) # 50
print(nums[1:4]) # [20, 30, 40]
print(nums[::-1]) # [50, 40, 30, 20, 10] reversed
```

Full Method Reference

Method / Syntax	What it does
`lst.append(x)`	Add a single item to the end - O(1) average
`lst.extend(iterable)`	Add all items from another iterable to the end
`lst.insert(i, x)`	Insert x at index i, shifting later elements - O(n)
`lst.remove(x)`	Remove the FIRST occurrence of value x - O(n)
`lst.pop(i=-1)`	Remove and return item at index i (default: last)
`lst.clear()`	Remove all items
`lst.index(x)`	Index of the first occurrence of x
`lst.count(x)`	Number of occurrences of x
`lst.sort(key=None, reverse=False)`	Sort the list IN PLACE
`lst.reverse()`	Reverse the list IN PLACE

`lst.copy()`	Shallow copy of the list

sorted() vs .sort()

```
nums = [5, 1, 4, 2]
new_list = sorted(nums) # returns a NEW sorted list, original unchanged
nums.sort() # sorts IN PLACE, returns None

words = ["banana", "kiwi", "apple"]
words.sort(key=len) # sort by string length
words.sort(key=len, reverse=True) # longest first
```

List Comprehensions

```
squares = [x*x for x in range(6)] # [0,1,4,9,16,25]
evens = [x for x in range(10) if x % 2 == 0] # filter: [0,2,4,6,8]
flat = [x for row in [[1,2],[3,4]] for x in row] # nested -> [1,2,3,4]
labeled = ["even" if x % 2 == 0 else "odd" for x in range(4)] # conditional
expression
```

How to read it: [expression for item in iterable if condition] — read the 'for' part first like a normal loop, then the optional 'if' filter, then the expression placed at the front.

Nested Lists (2D Lists / Matrices)

```
grid = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
print(grid[1][2]) # 6 -> row 1, column 2
for row in grid:
    for val in row:
        print(val, end=" ")
```

Shallow Copy vs Deep Copy

This is one of the most common sources of bugs. A shallow copy duplicates the outer list, but nested objects inside are still SHARED references.

```
import copy
original = [[1, 2], [3, 4]]

shallow = original.copy() # or list(original) or original[:]
shallow[0][0] = 99
print(original) # [[99, 2], [3, 4]] -> inner list was SHARED!

deep = copy.deepcopy(original)
deep[0][0] = 555
print(original) # unaffected -> deepcopy recursively copies nested objects
```

Unpacking & Star Expressions

```
a, b, c = [1, 2, 3] # basic unpacking
first, *rest = [1, 2, 3, 4] # first=1, rest=[2,3,4]
*init, last = [1, 2, 3, 4] # init=[1,2,3], last=4
a, *_, z = [1, 2, 3, 4, 5] # ignore middle values with _
```

Lists as Stack / Queue, and zip()

```
stack = []
stack.append(1); stack.append(2)
stack.pop() # LIFO behavior, O(1)

names = ["a", "b", "c"]
ages = [20, 25, 30]
for name, age in zip(names, ages): # pairs elements from multiple lists
    print(name, age)
```

NOTE: Lists work great as a Stack (append/pop from the end, both O(1)), but are a poor Queue (pop(0) is O(n) since everything must shift). Use collections.deque for queues - covered in Chapter 24.

7. Tuples — In Full Depth

A tuple is an ordered, **immutable** collection. Once created, its elements (and their positions) cannot be changed — though if a tuple holds a mutable object like a list, that inner object can still be modified.

Creation, Packing & Unpacking

```
point = (4, 5)
single = (4,) # NOTE the trailing comma - without it, (4) is just the int 4
packed = 1, 2, 3 # parentheses are optional when packing
a, b, c = packed # unpacking

print(point[0]) # 4
# point[0] = 9 -> TypeError: tuples cannot be modified
```

Methods (Only Two — Because Tuples are Immutable)

Method / Syntax	What it does
`t.count(x)`	Number of occurrences of x
`t.index(x)`	Index of the first occurrence of x

Named Tuples

collections.namedtuple creates tuple subclasses with named fields — combining the memory efficiency of a tuple with the readability of attribute access.

```
from collections import namedtuple
Point = namedtuple("Point", ["x", "y"])
p = Point(3, 4)
print(p.x, p.y) # 3 4
print(p[0], p[1]) # 3 4 -> still works like a regular tuple too
```

When to Use a Tuple vs a List

- Use a tuple for fixed, heterogeneous data that shouldn't change (e.g., a coordinate, an RGB color).
- Tuples are hashable (if their contents are), so they can be used as dictionary keys — lists cannot.
- Tuples are slightly faster to create and use less memory than lists.
- Use a list when you need to add/remove/reorder items.

8. Sets & Frozensets — In Full Depth

A set is an unordered collection of **unique, hashable** elements, implemented internally using a hash table (same family of structure as a dict). This gives average O(1) membership testing.

Creation & Membership

```
s1 = {1, 2, 3, 2, 1} # duplicates auto-removed -> {1, 2, 3}
s2 = set([3, 4, 5]) # build a set from a list
empty = set() # NOTE: {} creates an empty DICT, not a set!

print(2 in s1) # True - O(1) average lookup
```

Full Method Reference

Method / Syntax	What it does
`s.add(x)`	Add a single element
`s.remove(x)`	Remove x; raises KeyError if not present
`s.discard(x)`	Remove x if present; no error if missing
`s.pop()`	Remove and return an arbitrary element
`s.clear()`	Remove all elements
`s.union(other) / s | other`	All elements from both sets
`s.intersection(other) / s & other`	Elements common to both sets

`s.difference(other) / s - other`	Elements in s but not in other
`s.symmetric_difference(o) / s ^ o`	Elements in exactly one of the sets
`s.update(other)`	Add all elements of other into s (in place)
`s.intersection_update(other)`	Keep only elements also in other (in place)
`s.difference_update(other)`	Remove elements found in other (in place)
`s.issubset(other) / s <= other`	True if every element of s is in other
`s.issuperset(other) / s >= other`	True if s contains every element of other
`s.isdisjoint(other)`	True if the sets share NO elements

Set Comprehensions

```
squares = {x*x for x in range(6)} # {0, 1, 4, 9, 16, 25}
vowels_in = {ch for ch in "hello world" if ch in "aeiou"} # {'e', 'o'}
```

Frozenset — the Immutable Set

A frozenset behaves exactly like a set but cannot be modified after creation, which makes it hashable — so it can be used as a dictionary key or stored inside another set.

```
fs = frozenset([1, 2, 3])
# fs.add(4) -> AttributeError, frozensets have no add()

nested = {frozenset([1,2]), frozenset([3,4])} # a set of frozensets - legal!
d = {frozenset([1,2]): "pair A"} # frozenset as a dict key - legal!
```

NOTE: Sets are unordered — never rely on insertion order being preserved (unlike modern dicts, which do preserve insertion order). Use a set whenever you need fast 'have I seen this?' checks or to remove duplicates.

9. Dictionaries — In Full Depth

A dictionary stores key-value pairs and is implemented as a hash table, giving average O(1) lookup, insert, and delete. Since Python 3.7, dicts officially preserve insertion order.

Creation & Access

```
student = {"name": "Asha", "age": 21, "grade": "A"}
student2 = dict(name="Ravi", age=19) # alternate constructor

print(student["name"]) # Asha
# print(student["city"]) # KeyError! key does not exist
print(student.get("city")) # None - no error
print(student.get("city", "N/A")) # "N/A" - custom default

student["age"] = 22 # update existing key
student["city"] = "Pune" # add a new key
student.setdefault("grade", "F") # only sets if key is missing (no-op here)
```

Full Method Reference

Method / Syntax	What it does
`d.keys()`	View of all keys
`d.values()`	View of all values
`d.items()`	View of (key, value) pairs
`d.get(k, default)`	Safe lookup, returns default if key missing
`d.setdefault(k, default)`	Return d[k] if present, else insert default and return it
`d.update(other)`	Merge another dict (or key/value pairs) into d
`d.pop(k, default)`	Remove key k and return its value
`d.popitem()`	Remove and return the LAST inserted (key, value) pair
`d.clear()`	Remove all items
`d.copy()`	Shallow copy
`dict.fromkeys(seq, value)`	Build a new dict with keys from seq, all set to value

Iterating Over a Dictionary

```
for key in student: # iterates over KEYS by default
    print(key)
for key, value in student.items(): # iterates over (key, value) pairs
    print(key, "->", value)
for value in student.values():
    print(value)
```

Dict Comprehensions & Merging

```
squares = {x: x*x for x in range(5)} # {0:0, 1:1, 2:4, 3:9, 4:16}
filtered = {k:v for k,v in student.items() if isinstance(v, str)}

# Merging dicts (Python 3.9+)
d1 = {"a": 1, "b": 2}
d2 = {"b": 20, "c": 3}
merged = d1 | d2 # d2 values win on key conflict
d1 |= d2 # in-place merge update
```

Nested Dictionaries

```
users = {
    "alice": {"age": 30, "city": "NYC"},
    "bob": {"age": 25, "city": "LA"},
}
print(users["alice"]["city"]) # NYC
```

Specialized dict-like Tools (collections module)

Method / Syntax	What it does
`collections.defaultdict(int)`	Dict that auto-creates a default value for missing keys
`collections.Counter(iterable)`	Dict subclass for counting hashable items
`collections.OrderedDict()`	Dict that remembers insertion order (regular dict does too now)

```
from collections import defaultdict, Counter

graph = defaultdict(list)
graph["A"].append("B") # no KeyError even though "A" didn't exist yet

words = ["a","b","a","c","b","a"]
print(Counter(words)) # Counter({'a': 3, 'b': 2, 'c': 1})
print(Counter(words).most_common(1)) # [('a', 3)] -> most frequent item
```

NOTE: Dictionary keys must be hashable (immutable types like str, int, tuple-of-immutables). Lists and other dicts CANNOT be used as keys because they are mutable.

10. Functions — In Full Depth

A function is a reusable, named block of code that performs a specific task, optionally accepting inputs (parameters) and producing an output (return value).

```
def greet(name):
    return f"Hi {name}"

message = greet("Asha")
print(message) # Hi Asha
```

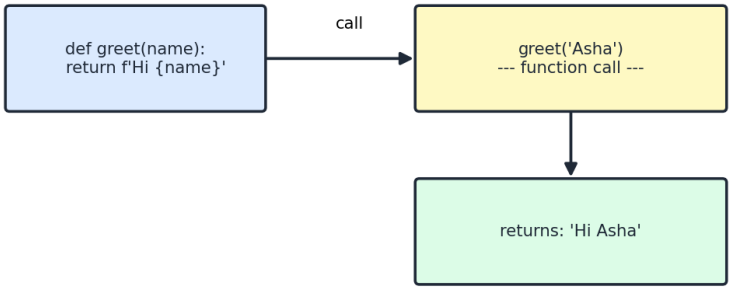

Figure 10.1 — A function call and its return value

Every Kind of Parameter

```
def demo(a, b, /, c, d, *, e, f):
    # a, b -> POSITIONAL-ONLY (before /)
    # c, d -> POSITIONAL-OR-KEYWORD (normal)
    # e, f -> KEYWORD-ONLY (after *)
    print(a, b, c, d, e, f)

demo(1, 2, 3, d=4, e=5, f=6) # valid
# demo(a=1, b=2, c=3, d=4, e=5, f=6) -> ERROR: a,b cannot be passed as
keywords
```

Default Arguments

```
def power(base, exponent=2):
    return base ** exponent

print(power(5)) # 25 -> exponent defaults to 2
print(power(2, 3)) # 8
print(power(exponent=3, base=2)) # 8 -> keyword args, order doesn't matter
```

NOTE: Never use a mutable default argument like def f(x, items=[]): — the SAME list is reused across every call, causing surprising bugs. Use 'items=None' and create a new list inside the function instead.

*args and **kwargs

```
def total(*numbers): # collects extra positional args into a tuple
    return sum(numbers)
print(total(1, 2, 3, 4)) # 10

def profile(**info): # collects extra keyword args into a dict
    for k, v in info.items():
        print(k, v)
profile(name="Asha", age=21)

# Unpacking when CALLING a function:
args = (1, 2, 3)
print(total(*args)) # unpacks tuple into positional args
kwargs = {"name": "Ravi", "age": 19}
profile(**kwargs) # unpacks dict into keyword args
```

Multiple Return Values

```
def min_max(nums):
    return min(nums), max(nums) # actually returns a tuple

lo, hi = min_max([4, 1, 9, 2])
print(lo, hi) # 1 9
```

Docstrings & Type Hints

```
def add(a: int, b: int) -> int:
    """Return the sum of two integers.

    Args:
        a: first number
        b: second number
    """
    return a + b

print(add.__doc__) # prints the docstring text
help(add) # shows signature + docstring
```

Type hints (a: int, -> int) do not actually enforce anything at runtime — Python remains dynamically typed — but they help tools (and humans) catch mistakes and improve editor autocompletion.

Scope — the LEGB Rule

When Python looks up a name, it searches in this order: **L**ocal -> **E**nclosing -> **G**lobal -> **B**uilt-in.

```
x = "global"

def outer():
    x = "enclosing"
    def inner():
        x = "local"
        print(x) # local
    inner()
    print(x) # enclosing

outer()
print(x) # global

def modify_global():
    global x
    x = "changed!" # without 'global', this would create a NEW local variable
instead
```

Lambda Functions

A lambda is a small, anonymous, single-expression function — most useful as a short, throwaway function passed into another function.

```
square = lambda x: x * x
print(square(5)) # 25

nums = [5, 1, 4, 2]
print(sorted(nums, key=lambda x: -x)) # [5, 4, 2, 1]
```

Higher-Order Functions: map, filter, reduce

```
nums = [1, 2, 3, 4, 5]

squared = list(map(lambda x: x*x, nums)) # [1,4,9,16,25]
evens = list(filter(lambda x: x % 2 == 0, nums)) # [2, 4]

from functools import reduce
total = reduce(lambda acc, x: acc + x, nums) # 15 -> cumulative reduction
product = reduce(lambda acc, x: acc * x, nums, 1) # 120 -> with starting value
1
```

NOTE: map() and filter() return ITERATOR objects in Python 3 (not lists) — wrap them in list(...) to see all results at once, or iterate them lazily with a for loop.

Function Annotations Summary & Recap Table

Concept	Syntax	Use case

Default arg	def f(x=5):	Optional parameter with fallback value
*args	def f(*args):	Accept any number of positional args
**kwargs	def f(**kwargs):	Accept any number of keyword args
Lambda	lambda x: x+1	Short, inline, anonymous function
Keyword-only	def f(*, x):	Force caller to use f(x=val)
Positional-only	def f(x, /):	Force caller to use f(val), no x=val

PART II

Intermediate & Advanced Python — OOP, errors, files, generators, decorators

11. Object-Oriented Programming — In Full Depth

OOP is a programming paradigm built around 'objects' — bundles of data (attributes) and behavior (methods). It is the foundation for writing large, maintainable programs and for understanding how Python's own built-in types are designed.

Classes, Objects, __init__ and self

```
class Dog:
    species = "Canine" # CLASS attribute - shared by all instances

    def __init__(self, name, breed):
        self.name = name # INSTANCE attribute - unique per object
        self.breed = breed

    def bark(self):
        return f"{self.name} says Woof!"

d1 = Dog("Rex", "Labrador")
d2 = Dog("Milo", "Pug")
print(d1.bark()) # Rex says Woof!
print(d1.species, d2.species) # Canine Canine -> shared class attribute
```

Line-by-line: __init__ is the constructor, automatically called when you create a new object. 'self' refers to the specific instance being created/used and must be the first parameter of every instance method.

Instance, Class & Static Methods

```
class Circle:
    pi = 3.14159

    def __init__(self, radius):
        self.radius = radius

    def area(self): # INSTANCE method - needs an object (self)
        return Circle.pi * self.radius ** 2

    @classmethod
    def from_diameter(cls, diameter): # CLASS method - receives the class
(cls), not an instance
        return cls(diameter / 2) # alternate constructor pattern

    @staticmethod
    def is_valid_radius(value): # STATIC method - no self or cls, just a
grouped utility function
        return value > 0

c = Circle.from_diameter(10)
print(c.radius) # 5.0
print(Circle.is_valid_radius(-2)) # False
```

Encapsulation & Name Mangling

```
class Account:
    def __init__(self, balance):
        self._balance = balance # _single underscore -> "protected"
(convention only)
        self.__pin = 1234 # __double underscore -> name-mangled, harder to
access

    def get_balance(self):
        return self._balance

a = Account(1000)
print(a._balance) # works, but signals "internal use only" by convention
# print(a.__pin) # AttributeError!
print(a._Account__pin) # 1234 -> name mangling: __pin becomes _Account__pin
internally
```

Properties — Pythonic Getters/Setters

```
class Temperature:
    def __init__(self, celsius):
        self._celsius = celsius

    @property
    def fahrenheit(self): # acts like an attribute, but computed on read
        return self._celsius * 9/5 + 32

    @fahrenheit.setter
    def fahrenheit(self, value): # lets you "set" a computed property
        self._celsius = (value - 32) * 5/9

t = Temperature(25)
print(t.fahrenheit) # 77.0 -> looks like an attribute, runs a method
t.fahrenheit = 98.6
print(t._celsius) # 37.0
```

Inheritance — All Forms

- Single — one child inherits from one parent.
- Multilevel — A -> B -> C, a chain of inheritance.
- Hierarchical — multiple children inherit from the same parent.
- Multiple — a child inherits from more than one parent class at once.
- Hybrid — a combination of the above forms.

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return "Some sound"

class Dog(Animal): # single inheritance
    def speak(self): # method OVERRIDING
        return f"{self.name} says Woof"

class Puppy(Dog): # multilevel inheritance (Animal -> Dog -> Puppy)
    def speak(self):
        original = super().speak() # call the PARENT's version too
        return original + " (in a tiny voice)"

class Swimmer:
    def swim(self):
        return "Swimming!"

class Duck(Animal, Swimmer): # MULTIPLE inheritance - two parents
    pass

d = Puppy("Rex")
print(d.speak()) # Rex says Woof (in a tiny voice)
duck = Duck("Donald")
print(duck.swim()) # Swimming! -> inherited from Swimmer
```

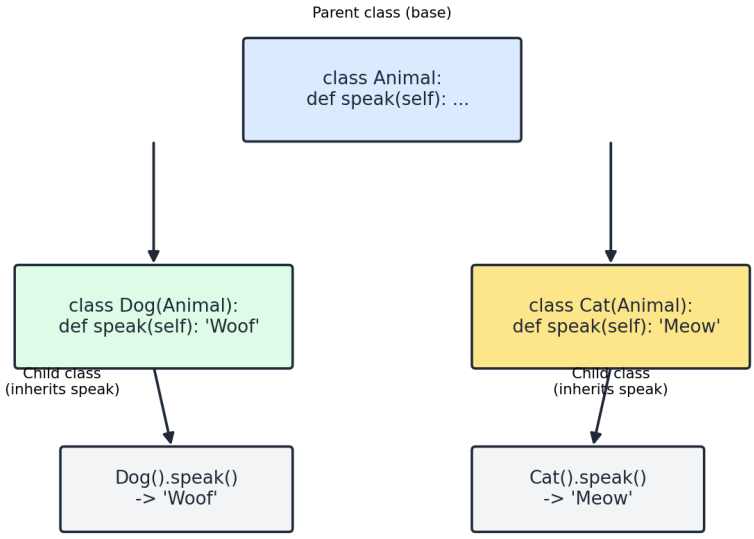

Figure 11.1 — Dog and Cat both inherit shared behavior from Animal

Polymorphism & Duck Typing

```
class Cat(Animal):
    def speak(self):
        return f"{self.name} says Meow"

for a in [Dog("Rex"), Cat("Tom")]:
    print(a.speak()) # same method name, different behavior per class

# Duck typing: "if it walks like a duck and quacks like a duck..."
class Robot:
    def speak(self):
        return "Beep boop"

for thing in [Dog("R2"), Robot()]:
    print(thing.speak()) # works even though Robot is unrelated to Animal
```

Abstraction & Abstract Base Classes

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass # no implementation - forces subclasses to provide one

class Square(Shape):
    def __init__(self, side):
        self.side = side
    def area(self):
        return self.side ** 2

# s = Shape() -> TypeError: cannot instantiate an abstract class
sq = Square(4)
print(sq.area()) # 16
```

Magic / Dunder Methods (Operator Overloading)

Method / Syntax	What it does
`__init__(self, ...)`	Constructor, runs on object creation
`__str__(self)`	Human-readable string, used by print() and str()
`__repr__(self)`	Unambiguous developer-facing representation
`__len__(self)`	Defines behavior of len(obj)
`__eq__(self, other)`	Defines behavior of == comparisons
`__lt__ / __gt__ / __le__ / __ge__`	Defines <, >, <=, >= comparisons

`__add__(self, other)`	Defines behavior of the + operator
`__getitem__(self, key)`	Defines behavior of obj[key]
`__iter__ / __next__`	Makes an object iterable (see Chapter 16)
`__del__(self)`	Destructor, runs when the object is garbage collected

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __add__(self, other): # enables vec1 + vec2
        return Vector(self.x + other.x, self.y + other.y)
    def __repr__(self):
        return f"Vector({self.x}, {self.y})"

v1, v2 = Vector(1, 2), Vector(3, 4)
print(v1 + v2) # Vector(4, 6) -> + operator now works on custom objects
```

Method Overriding vs Overloading

- Overriding (fully supported) — a child class redefines a method that already exists in the parent.
- Overloading (not natively supported) — Python doesn't allow multiple methods with the same name but different parameter lists. Instead, achieve similar behavior with default arguments or *args/**kwargs.

> **NOTE:** 'Composition over inheritance' is a common design principle: instead of inheriting from a class just to reuse code, consider storing an instance of that class as an attribute (e.g., a Car HAS an Engine, rather than a Car IS an Engine).

12. Exception Handling & Context Managers

Errors are inevitable - a file might be missing, user input might be invalid, or a calculation might be undefined. Exception handling lets your program respond gracefully instead of crashing.

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
except ValueError:
    print("That was not a valid number!")
except ZeroDivisionError:
    print("Cannot divide by zero!")
else:
    print("Result:", result) # runs only if NO exception occurred
finally:
    print("Done attempting the operation.") # ALWAYS runs
```

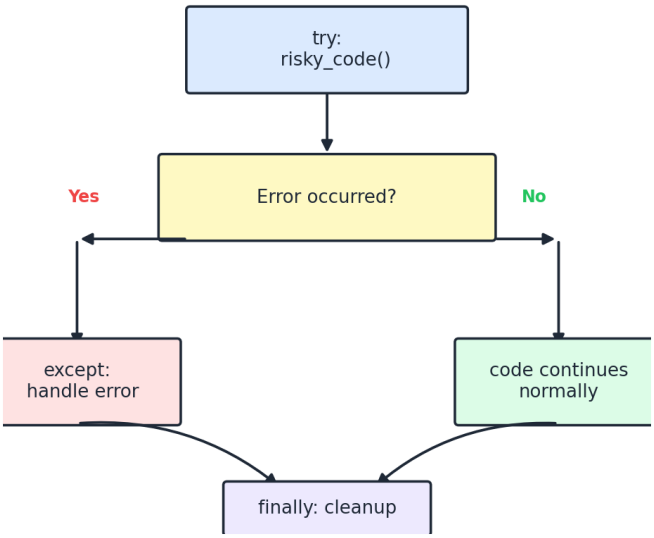

Figure 12.1 — try / except / finally execution flow

Catching Multiple Exceptions & the Exception Hierarchy

```
try:
    risky()
except (ValueError, TypeError) as e: # catch several types in one block
    print("Input problem:", e)
except Exception as e: # generic catch-all (use sparingly!)
    print("Something else went wrong:", e)
```

- BaseException is the root of all exceptions.
- Exception is the base class for almost everything you should catch.
- Common built-ins: ValueError, TypeError, KeyError, IndexError, ZeroDivisionError, FileNotFoundError, AttributeError.
- Catching the broad 'Exception' hides bugs - catch the SPECIFIC exception you expect whenever possible.

Raising Exceptions & Custom Exceptions

```
def set_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    return age

class InsufficientFundsError(Exception): # custom exception - just inherit
from Exception
    pass

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFundsError(f"Cannot withdraw {amount}, balance is
{balance}")
    return balance - amount
```

assert Statements

```
def divide(a, b):
    assert b != 0, "b must not be zero" # raises AssertionError if condition
is False
    return a / b
```

Context Managers (the with statement)

A context manager guarantees setup and cleanup code runs correctly, even if an error occurs - most commonly seen with file handling.

```
class Timer:
    def __enter__(self):
        import time
        self.start = time.time()
        return self
    def __exit__(self, exc_type, exc_val, exc_tb):
        print("Elapsed:", __import__("time").time() - self.start)

with Timer():
    total = sum(range(1000000)) # __enter__ runs first, __exit__ runs after,
even on error
```

NOTE: Always prefer specific exceptions over a bare 'except:' (which even catches Ctrl+C). And prefer 'with open(...) as f:' over manually calling f.close(), since 'with' guarantees cleanup.

13. File Handling & the os/pathlib modules

Python reads from and writes to files using the built-in open() function, which returns a file object.

```
# Writing (overwrites the file if it exists)
with open("notes.txt", "w") as f:
    f.write("Line 1\n")
    f.writelines(["Line 2\n", "Line 3\n"])

# Reading the whole file
with open("notes.txt", "r") as f:
    content = f.read()

# Reading line by line (memory efficient for big files)
with open("notes.txt", "r") as f:
    for line in f:
        print(line.strip())

# Reading all lines into a list
with open("notes.txt", "r") as f:
    lines = f.readlines()

# Appending
with open("notes.txt", "a") as f:
    f.write("Line 4\n")
```

File Modes

Mode	Meaning
'r'	Read (default); error if file doesn't exist
'w'	Write; creates new or OVERWRITES existing file
'a'	Append; creates new or adds to end of existing file
'x'	Exclusive creation; errors if file already exists
'r+'	Read and write, file must exist
'b'	Binary mode suffix, e.g. 'rb', 'wb' for non-text files

seek() and tell()

```
with open("notes.txt", "r") as f:
    print(f.tell()) # 0 - current cursor position
    f.seek(5) # move cursor to byte offset 5
    print(f.read(10)) # read only the next 10 characters
```

os and pathlib for File System Operations

```
import os
from pathlib import Path

print(os.getcwd()) # current working directory
print(os.listdir(".")) # list files in a directory
os.makedirs("new_folder", exist_ok=True)
print(os.path.exists("notes.txt")) # True/False

p = Path("data/notes.txt") # modern, object-oriented path handling
print(p.name, p.suffix, p.parent)
print(p.exists())
```

14. Modules, Packages & the Standard Library Tour

Importing

```
import math # import the whole module
from math import sqrt, pi # import specific names
import numpy as np # import with an alias
from math import * # import everything (generally discouraged)

print(math.sqrt(16)) # 4.0
```

__name__ == "__main__"

```
# my_module.py
def main():
    print("Running directly")

if __name__ == "__main__":
    main() # only runs when this file is executed directly, NOT when imported
```

Creating Your Own Modules & Packages

- A module is simply any .py file - its name (minus .py) becomes the import name.
- A package is a folder containing an __init__.py file (can be empty) plus other modules, allowing 'import mypackage.module'.
- pip install <name> installs third-party packages from PyPI; 'pip freeze' lists installed packages.
- Virtual environments (python -m venv venv) isolate project dependencies from your system Python.

Useful Standard Library Modules

Module	Purpose
math	Mathematical functions and constants

random	Random number generation
datetime	Dates and times
collections	Specialized containers (Counter, deque, defaultdict, namedtuple)
itertools	Fast, memory-efficient iterator building blocks
functools	Higher-order functions (reduce, lru_cache, partial)
re	Regular expressions
os / sys	Operating system and interpreter interaction
json	Encoding/decoding JSON data

A Few itertools & functools Highlights

```
from itertools import permutations, combinations, product, chain
print(list(permutations([1,2,3], 2))) # all ordered pairs
print(list(combinations([1,2,3], 2))) # all unordered pairs
print(list(product([1,2],[3,4]))) # cartesian product
print(list(chain([1,2],[3,4]))) # flatten/chain iterables together

from functools import lru_cache
@lru_cache(maxsize=None) # auto-memoizes results (see Recursion chapter)
def fib(n):
    return n if n <= 1 else fib(n-1) + fib(n-2)
```

15. Comprehensions (List / Set / Dict / Generator)

Comprehensions provide a concise, readable way to build collections in a single expression instead of writing a multi-line loop with .append().

```
# List comprehension
squares = [x*x for x in range(6)]

# Set comprehension
unique_lengths = {len(w) for w in ["hi","bye","ok"]}

# Dict comprehension
square_map = {x: x*x for x in range(5)}

# Generator expression - LAZY, uses () instead of []
gen = (x*x for x in range(1000000)) # does NOT compute all values immediately
print(next(gen)) # 0 -> computes just the first value

# Nested comprehension (matrix transpose)
matrix = [[1,2,3], [4,5,6]]
transposed = [[row[i] for row in matrix] for i in range(3)]
# [[1,4],[2,5],[3,6]]
```

NOTE: Generator expressions use almost no memory regardless of size, because values are produced one at a time on demand instead of being stored in a list. Use them whenever you only need to iterate ONCE over the results.

16. Iterators & Generators

The Iterator Protocol

Anything you can loop over (list, string, range, dict) is **iterable** - it has an __iter__ method. Calling iter() on it produces an **iterator**, an object with a __next__ method that produces values one at a time until it raises StopIteration.

```
nums = [10, 20, 30]
it = iter(nums) # get an iterator from the list
print(next(it)) # 10
print(next(it)) # 20
print(next(it)) # 30
# print(next(it)) -> StopIteration raised, no more items

class Countdown: # a custom iterator
    def __init__(self, start):
        self.current = start
    def __iter__(self):
        return self
    def __next__(self):
        if self.current <= 0:
            raise StopIteration
        self.current -= 1
        return self.current + 1

for n in Countdown(3):
    print(n) # 3 2 1
```

Generators (yield)

A generator function produces a sequence of values lazily, one at a time, using 'yield' instead of 'return'. The function's state is automatically paused and resumed between calls - saving memory since values aren't all stored at once.

```
def count_up_to(n):
    i = 1
    while i <= n:
        yield i # pauses here, returns i, resumes from here on next() call
        i += 1

gen = count_up_to(3)
print(next(gen)) # 1
print(next(gen)) # 2
for num in count_up_to(5): # generators work directly in for loops too
    print(num)
```

yield vs return

- return exits the function permanently and sends back ONE value.
- yield pauses the function, sends back ONE value, but remembers its state to continue later.
- A function containing 'yield' becomes a generator function - calling it returns a generator object immediately, without running any code yet.

The itertools Module for Advanced Iteration

- itertools.count(start) - infinite counter
- itertools.cycle(iterable) - repeats a sequence forever
- itertools.islice(iterable, n) - take the first n items from any iterable, even infinite ones

17. Recursion — In Full Depth

Recursion is when a function calls itself to solve a smaller version of the same problem, until it reaches a 'base case' that stops the recursion. It is central to trees, graphs, sorting (merge/quick sort), and backtracking algorithms.

Anatomy of a Recursive Function

- Base case - the simplest input, handled directly without further recursion (stops infinite recursion).
- Recursive case - breaks the problem into a smaller version of itself, and calls the function again.

```
def factorial(n):
    if n == 0 or n == 1: # base case
        return 1
    return n * factorial(n - 1) # recursive case

print(factorial(4)) # 24
```

Line-by-line trace for factorial(4):

- factorial(4) calls 4 * factorial(3)
- factorial(3) calls 3 * factorial(2)
- factorial(2) calls 2 * factorial(1)

- factorial(1) hits the base case and returns 1
- Calls 'unwind': 2*1=2, then 3*2=6, then 4*6=24

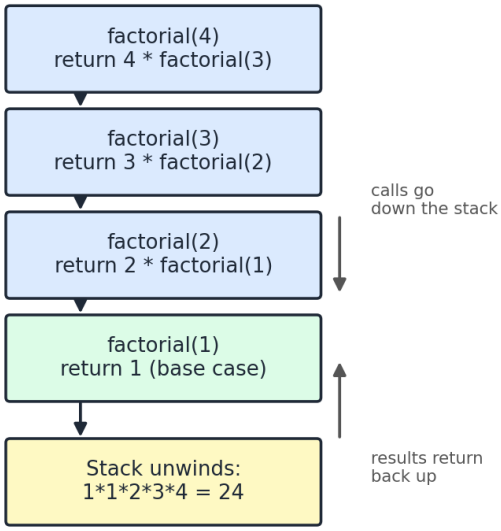

Figure 17.1 — The recursive call stack building up and unwinding

Direct vs Indirect Recursion

- Direct recursion - a function calls itself directly (factorial above).
- Indirect recursion - function A calls function B, which calls function A again.

Tail Recursion

A tail-recursive call is the LAST operation in a function, with no pending work after it returns. Some languages optimize this to avoid growing the call stack - Python does NOT perform this optimization, so deep tail recursion in Python can still hit the recursion limit.

```
def factorial_tail(n, acc=1):
    if n <= 1:
        return acc
    return factorial_tail(n - 1, acc * n) # the recursive call is the very
last action
```

Memoization (Caching Recursive Results)

```
from functools import lru_cache

@lru_cache(maxsize=None)
def fib(n):
    if n <= 1:
        return n
    return fib(n-1) + fib(n-2) # repeated calls are now cached automatically
```

Classic Recursion Examples

```
def sum_list(lst): # sum of a list
    if not lst:
        return 0
    return lst[0] + sum_list(lst[1:])

def power(base, exp): # exponentiation
    if exp == 0:
        return 1
    return base * power(base, exp - 1)

def gcd(a, b): # greatest common divisor (Euclidean algorithm)
    if b == 0:
        return a
    return gcd(b, a % b)

def reverse_string(s): # reverse a string
    if len(s) <= 1:
        return s
    return reverse_string(s[1:]) + s[0]

def hanoi(n, source, target, aux): # Tower of Hanoi
    if n == 1:
        print(f"Move disk 1 from {source} to {target}")
        return
    hanoi(n-1, source, aux, target)
    print(f"Move disk {n} from {source} to {target}")
    hanoi(n-1, aux, target, source)

def permute(s, path=""): # all permutations of a string
    if not s:
        print(path)
        return
    for i in range(len(s)):
        permute(s[:i] + s[i+1:], path + s[i])
```

NOTE: Every recursive function MUST have a base case, or it will recurse forever and crash with a RecursionError (Python's default recursion limit is ~1000 calls deep).

18. Decorators & Closures

Closures

A closure is a function that 'remembers' variables from its enclosing scope even after that outer function has finished executing.

```
def make_multiplier(factor):
    def multiplier(x):
        return x * factor # 'factor' is remembered from the enclosing scope
    return multiplier

times3 = make_multiplier(3)
print(times3(10)) # 30 -> still remembers factor=3
```

Decorators

A decorator is a function that takes another function and extends/modifies its behavior without permanently changing its source code. The @ syntax is just shorthand.

```
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print("Before the function runs")
        result = func(*args, **kwargs)
        print("After the function runs")
        return result
    return wrapper

@my_decorator # equivalent to: say_hello = my_decorator(say_hello)
def say_hello(name):
    print(f"Hello, {name}!")

say_hello("Asha")
# Before the function runs
# Hello, Asha!
# After the function runs
```

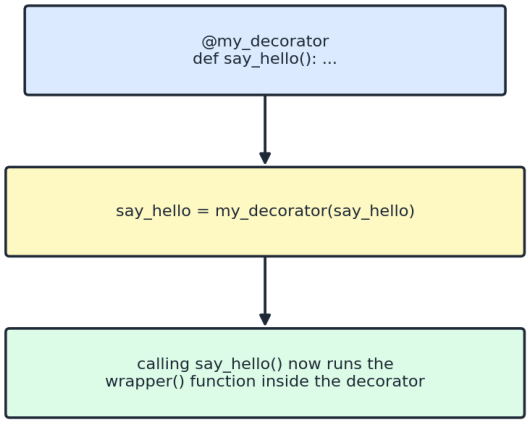

Figure 18.1 — How @decorator syntax rewires a function call

Common Built-in Decorators

- @staticmethod and @classmethod - covered in OOP (Chapter 11).
- @property - turns a method into a computed attribute (Chapter 11).
- @lru_cache - automatic memoization for recursive/expensive functions (Chapter 17).
- @functools.wraps - preserves a wrapped function's name/docstring when writing your own decorators.

```
from functools import wraps

def logger(func):
    @wraps(func) # keeps func.__name__ correct for debugging
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        return func(*args, **kwargs)
    return wrapper

@logger
def add(a, b):
    return a + b

print(add(2, 3)) # Calling add -> 5
```

19. Regular Expressions (re module)

Regular expressions describe text patterns for searching, matching, and replacing strings.

```
import re

text = "Contact: asha@example.com or call 9876543210"

email = re.search(r"[\w.]+@[\w.]+", text)
print(email.group()) # asha@example.com

phones = re.findall(r"\d{10}", text)
print(phones) # ['9876543210']

cleaned = re.sub(r"\d", "#", text) # replace every digit with #
print(cleaned)

if re.match(r"Contact", text): # match() only checks the START of the string
    print("Starts with Contact")
```

Common Pattern Symbols

Symbol	Meaning
\d	Any digit (0-9)

\w	Any word character (letter/digit/underscore)
\s	Any whitespace character
.	Any character except newline
*	Zero or more of the previous
+	One or more of the previous
?	Zero or one of the previous (optional)
{n}	Exactly n repetitions
^ / $	Start / end of string
[abc]	Any one of a, b, or c

NOTE: re.search() finds a match anywhere in the string; re.match() only checks the beginning; re.findall() returns ALL matches as a list; re.sub() replaces matches.

PART III

Data Structures & Algorithms — complete coverage for interviews

20. Big-O Notation & Complexity Analysis

Big-O notation describes how the running time (or memory) of an algorithm grows as the input size 'n' grows. It lets us compare algorithms independent of hardware or programming language.

Notation	Name	Example
O(1)	Constant	Accessing a list element by index
O(log n)	Logarithmic	Binary search
O(n)	Linear	Linear search, single loop
O(n log n)	Linearithmic	Merge sort, quick sort (avg), heap sort
O(n^2)	Quadratic	Bubble/selection/insertion sort, nested loops
O(2^n)	Exponential	Naive recursive Fibonacci, subsets
O(n!)	Factorial	Brute-force permutations (Traveling Salesman)

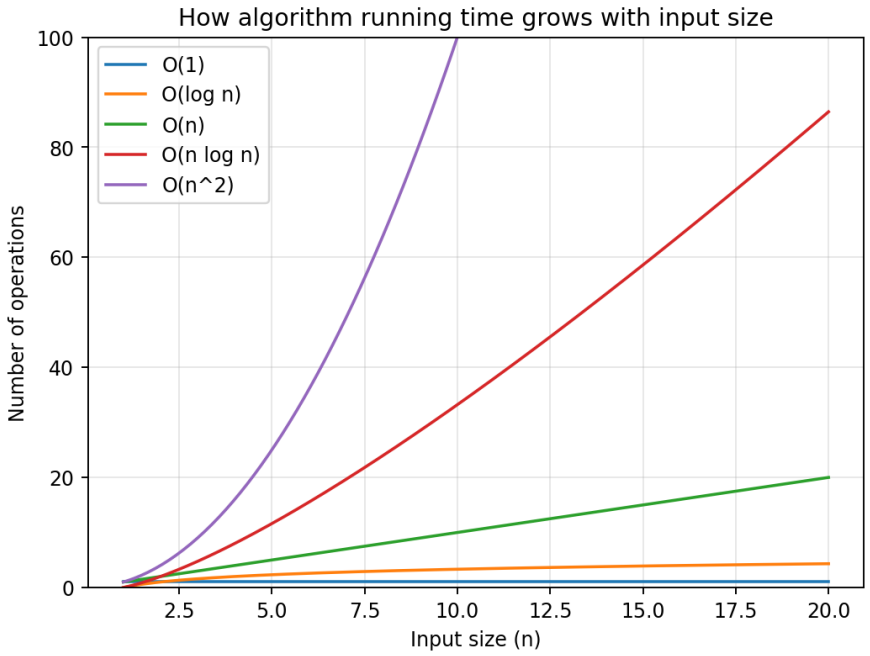

Figure 20.1 — How operation count grows with input size

Time Complexity vs Space Complexity

- Time complexity — how the NUMBER OF OPERATIONS grows with input size.
- Space complexity — how the EXTRA MEMORY used grows with input size (not counting the input itself).
- Recursive functions consume O(depth) extra space on the call stack, even with no extra data structures.

Best, Average & Worst Case

- Best case — the most favorable input (e.g., already-sorted array for insertion sort: O(n)).
- Average case — expected performance over typical/random inputs.
- Worst case — the most unfavorable input; this is usually what 'Big-O' refers to by default.

Amortized Analysis

Some operations are usually cheap but occasionally expensive — like a dynamic array doubling its size when full. Even though that resize is O(n), it happens rarely enough that the AVERAGE cost per append, over many operations, is still O(1) — called amortized O(1).

> **NOTE:** Quick analysis tips: a single loop over n items -> O(n). Nested loops over the same n items -> O(n^2). Halving input each step -> O(log n). A loop that calls a function with its own loop inside -> multiply their complexities.

21. Arrays / Lists as a Data Structure

An array is a collection of elements stored at contiguous memory locations, accessible directly by index. Python's list behaves like a dynamic array — it grows/shrinks automatically by over-allocating memory behind the scenes.

Operation	Time Complexity	Why
Access by index	O(1)	Direct memory address calculation
Search (unsorted)	O(n)	Must check elements one by one
Search (sorted, binary search)	O(log n)	Halve the search space each step
Insert/Delete at end	O(1) avg	No shifting needed (amortized)
Insert/Delete at start/middle	O(n)	Remaining elements must shift

Two-Pointer Technique

Two pointers move through a (usually sorted) array from different positions, narrowing in on an answer in a single O(n) pass — instead of the O(n^2) brute force of checking every pair.

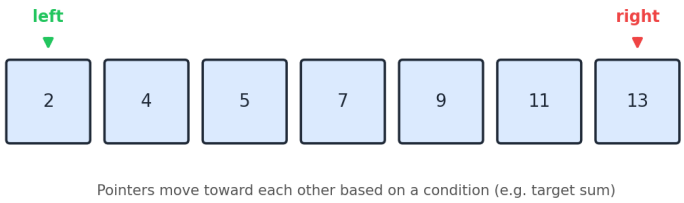

Figure 21.1 — Two pointers starting at opposite ends

```
def has_pair_with_sum(arr, target): # arr must be SORTED
    left, right = 0, len(arr) - 1
    while left < right:
        current = arr[left] + arr[right]
        if current == target:
            return True
        elif current < target:
            left += 1 # need a bigger sum -> move left pointer right
        else:
            right -= 1 # need a smaller sum -> move right pointer left
    return False
```

Sliding Window Technique

A sliding window maintains a running range [start, end] over an array/string, expanding and shrinking it to avoid recomputing results from scratch — turning many O(n^2) problems into O(n).

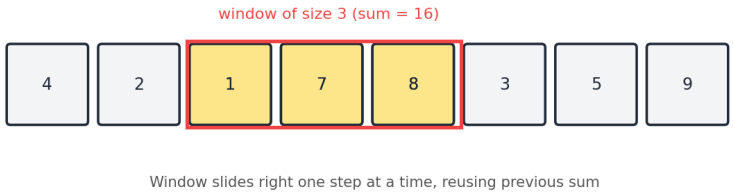

Figure 21.2 — A fixed-size window sliding across an array

```
def max_sum_subarray(arr, k): # max sum of any window of size k
    window_sum = sum(arr[:k])
    max_sum = window_sum
    for i in range(k, len(arr)):
        window_sum += arr[i] - arr[i-k] # add new element, remove the one
leaving the window
        max_sum = max(max_sum, window_sum)
    return max_sum
```

Prefix Sum

Precomputing cumulative sums lets you answer 'sum of any range' queries in O(1) after an O(n) setup, instead of recomputing each range sum from scratch.

```
def build_prefix_sums(arr):
    prefix = [0] * (len(arr) + 1)
    for i, val in enumerate(arr):
        prefix[i+1] = prefix[i] + val
    return prefix

prefix = build_prefix_sums([2, 4, 6, 8, 10])
# sum of arr[1:4] (4+6+8=18):
print(prefix[4] - prefix[1]) # 18
```

22. Linked Lists (Singly, Doubly, Circular)

A linked list is a linear data structure where each element ('node') stores its data plus a reference (pointer) to the next node. Nodes are NOT stored in contiguous memory, unlike arrays.

Singly Linked List

Figure 22.1 — A singly linked list: each node points only forward

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def append(self, data): # insert at the END - O(n)
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        current = self.head
        while current.next:
            current = current.next
        current.next = new_node

    def prepend(self, data): # insert at the START - O(1)
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

    def delete(self, key): # delete by value - O(n)
        current = self.head
        if current and current.data == key:
            self.head = current.next
            return
        prev = None
        while current and current.data != key:
            prev = current
            current = current.next
        if current:
            prev.next = current.next

    def reverse(self): # reverse the list IN PLACE - O(n)
        prev = None
        current = self.head
        while current:
            next_node = current.next
            current.next = prev
            prev = current
            current = next_node
        self.head = prev
```

Doubly Linked List

Each node also stores a pointer to the PREVIOUS node, allowing traversal in both directions and O(1) deletion when you already hold a reference to the node.

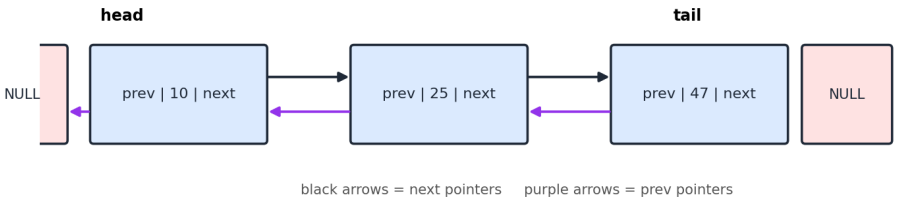

Figure 22.2 — A doubly linked list with next and prev pointers

```
class DNode:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None
```

Circular Linked List

The last node points back to the first node instead of None, forming a loop — useful for round-robin scheduling or repeating playlists.

Detecting a Cycle — Floyd's Algorithm (Slow/Fast Pointers)

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next # moves 1 step
        fast = fast.next.next # moves 2 steps
        if slow is fast: # they meet -> there IS a cycle
            return True
    return False # fast reached the end -> no cycle
```

Arrays vs Linked Lists

Feature	Array / List	Linked List
Memory layout	Contiguous	Scattered (linked via pointers)
Access by index	O(1)	O(n) - must traverse
Insert/Delete at start	O(n)	O(1)
Extra memory per element	None	1-2 pointers per node

23. Stacks

A stack is a linear data structure that follows **LIFO** — Last In, First Out. Think of a stack of plates: you add (push) and remove (pop) from the top only.

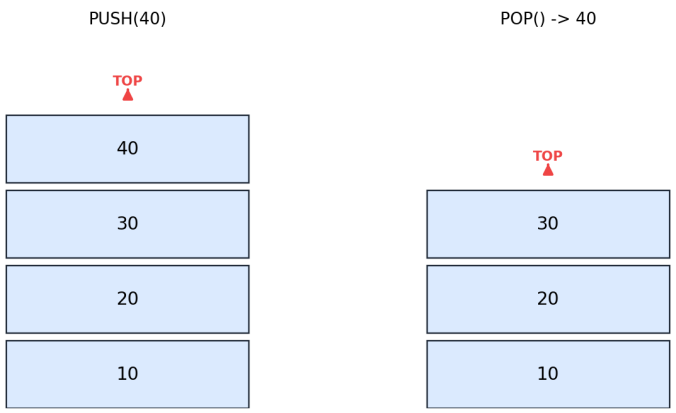

Figure 23.1 — push() adds to the top; pop() removes from the top

```
stack = []
stack.append(10) # push -> [10]
stack.append(20) # push -> [10, 20]
stack.append(30) # push -> [10, 20, 30]
print(stack.pop()) # pop -> removes & returns 30
print(stack) # [10, 20]
print(stack[-1]) # peek -> 20 (top element, without removing)
print(len(stack) == 0) # is_empty check
```

Real-World Applications

- Undo/redo functionality in editors.
- Browser back-button history.
- Function call stack (how recursion works internally).
- Balanced parentheses / bracket matching.
- Converting infix expressions to postfix, and evaluating postfix expressions.

Example: Balanced Parentheses

```
def is_balanced(expr):
    stack = []
    pairs = {")": "(", "]": "[", "}": "{"}
    for ch in expr:
        if ch in "([{":
            stack.append(ch)
        elif ch in ")]}":
            if not stack or stack.pop() != pairs[ch]:
                return False
    return len(stack) == 0

print(is_balanced("{[()]}")) # True
print(is_balanced("{[(])}")) # False
```

24. Queues (Linear, Circular, Deque, Priority Queue)

A queue follows **FIFO** — First In, First Out, like a line at a ticket counter.

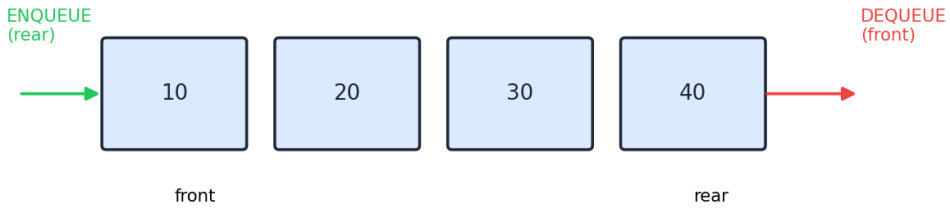

Figure 24.1 — Items enter at the rear and leave from the front

```
from collections import deque

queue = deque()
queue.append(10) # enqueue
queue.append(20)
queue.append(30)
print(queue.popleft()) # dequeue -> 10
print(queue) # deque([20, 30])
```

NOTE: Use collections.deque instead of a plain list for queues - popping from the front of a list is O(n), while deque.popleft() is O(1) (deque is a doubly linked list internally).

Circular Queue

A circular queue reuses freed-up space at the front by wrapping the rear pointer back to index 0 once it reaches the end of a fixed-size array — avoiding the wasted space of a simple array queue.

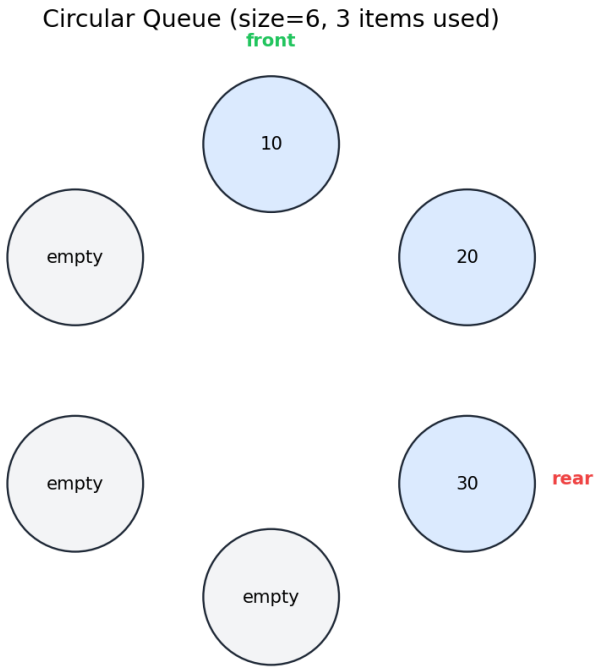

Figure 24.2 — A circular queue wraps around when it reaches the end

Deque (Double-Ended Queue)

A deque allows O(1) insertion and removal from BOTH ends, making it strictly more flexible than a stack or a simple queue.

```
from collections import deque
dq = deque([2, 3, 4])
dq.appendleft(1) # [1, 2, 3, 4]
dq.append(5) # [1, 2, 3, 4, 5]
dq.popleft() # removes 1
dq.pop() # removes 5
```

Priority Queue (heapq module)

A priority queue serves the HIGHEST (or lowest) priority element first, regardless of insertion order. Python's heapq module implements an efficient min-heap-based priority queue.

```
import heapq
pq = []
heapq.heappush(pq, 5)
heapq.heappush(pq, 1)
heapq.heappush(pq, 3)
print(heapq.heappop(pq)) # 1 - smallest element comes out first
print(heapq.heappop(pq)) # 3
```

- Real-world uses for queues: task scheduling, printer queues, breadth-first search (BFS).

- Real-world uses for priority queues: Dijkstra's algorithm, task schedulers with priorities, the A* pathfinding algorithm.

25. Searching Algorithms

Linear Search — O(n)

Checks every element one by one from the start until it finds the target or reaches the end. Works on both sorted and unsorted data.

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1
```

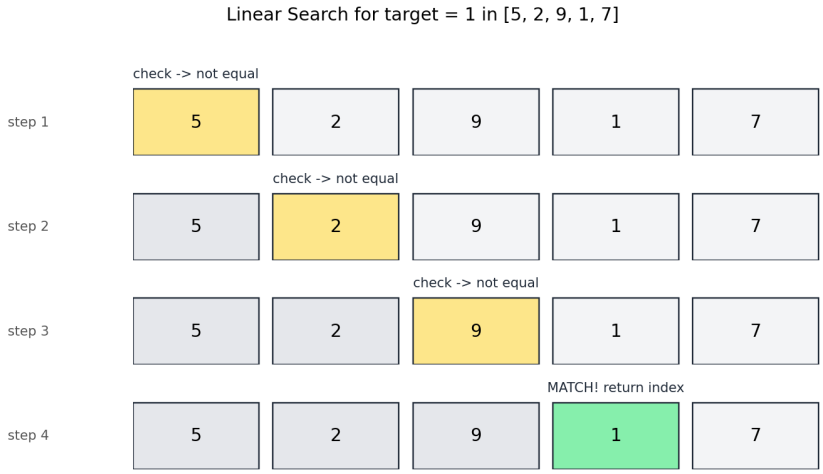

Figure 25.1 — Linear search checks each box left to right

Binary Search — O(log n)

Works only on **sorted** data. Repeatedly checks the middle element and eliminates half of the remaining elements each time.

```python
def binary_search(arr, target): # ITERATIVE version
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1

def binary_search_recursive(arr, target, lo, hi): # RECURSIVE version
    if lo > hi:
        return -1
    mid = (lo + hi) // 2
    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search_recursive(arr, target, mid + 1, hi)
    else:
        return binary_search_recursive(arr, target, lo, mid - 1)
```

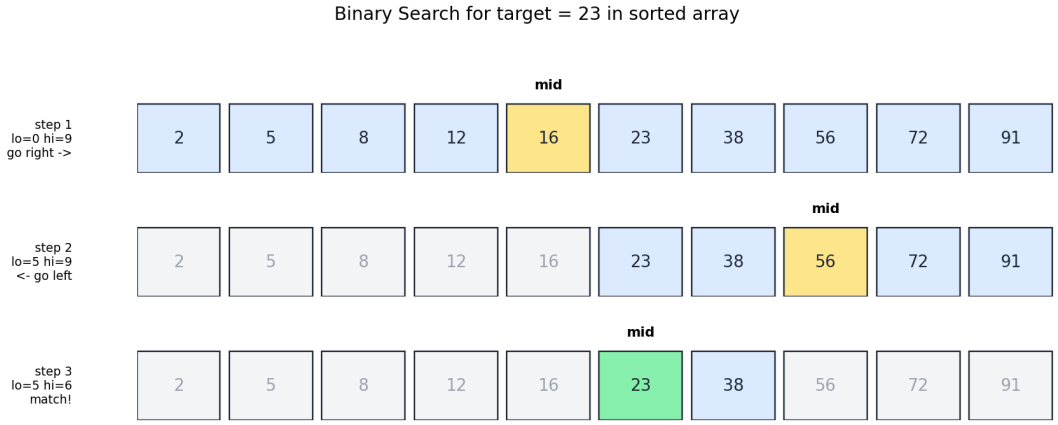

Figure 25.2 — Binary search narrows the range in half each step

Interpolation Search — O(log log n) average

An improvement on binary search for uniformly distributed sorted data: instead of always checking the middle, it estimates WHERE the target likely is, similar to how you'd open a dictionary near 'S' rather than the exact middle when looking up the word 'Snake'.

```
def interpolation_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi and arr[lo] <= target <= arr[hi]:
        if arr[hi] == arr[lo]:
            pos = lo
        else:
            pos = lo + (hi - lo) * (target - arr[lo]) // (arr[hi] - arr[lo])
        if arr[pos] == target:
            return pos
        elif arr[pos] < target:
            lo = pos + 1
        else:
            hi = pos - 1
    return -1
```

NOTE: Why O(log n) for binary search? Each comparison cuts the search space in half: 1000 -> 500 -> 250 -> ... -> 1 in about 10 steps.

26. Sorting Algorithms

Bubble Sort — O(n^2)

Repeatedly steps through the list, compares adjacent elements, and swaps them if out of order. After each pass, the largest unsorted element 'bubbles up' to its correct position.

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        for j in range(n - 1 - i):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
```

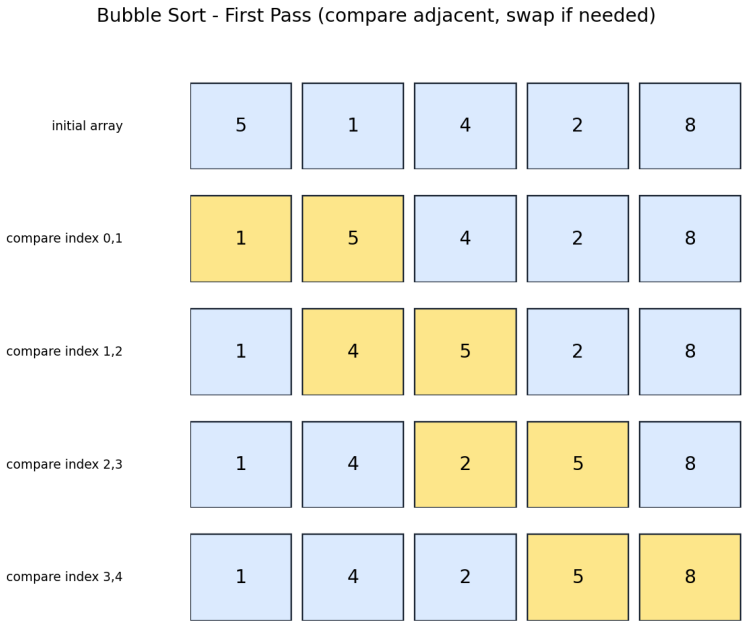

Figure 26.1 — One pass of bubble sort

Selection Sort — O(n^2)

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

Insertion Sort — O(n^2)

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key
    return arr
```

Merge Sort — O(n log n)

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:]); result.extend(right[j:])
    return result
```

Quick Sort — O(n log n) average

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    mid = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + mid + quick_sort(right)
```

Heap Sort — O(n log n)

Builds a max-heap from the array, then repeatedly swaps the root (largest element) with the last item and 'sinks' the new root down to restore the heap property.

```
import heapq
def heap_sort(arr):
    heapq.heapify(arr) # rearranges arr into a valid min-heap in O(n)
    return [heapq.heappop(arr) for _ in range(len(arr))] # pop smallest
repeatedly
```

Counting Sort — O(n + k) (non-comparison sort)

Works only for integers in a known, small range [0, k]. Counts occurrences of each value, then rebuilds the sorted array — faster than O(n log n) when k is small.

```
def counting_sort(arr, max_val):
    counts = [0] * (max_val + 1)
    for num in arr:
        counts[num] += 1
    result = []
    for value, count in enumerate(counts):
        result.extend([value] * count)
    return result
```

Comparison Table

Algorithm	Best	Average	Worst	Space	Stable?
Bubble Sort	O(n)	O(n^2)	O(n^2)	O(1)	Yes
Selection Sort	O(n^2)	O(n^2)	O(n^2)	O(1)	No
Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1)	Yes
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes
Quick Sort	O(n log n)	O(n log n)	O(n^2)	O(log n)	No
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	No
Counting Sort	O(n+k)	O(n+k)	O(n+k)	O(k)	Yes

27. Hashing & Hash Tables

A hash table stores key-value pairs and uses a hash function to convert a key into an index in an internal array, allowing average O(1) insert, lookup, and delete. Python's dict and set are both implemented using hash tables.

```
def count_frequency(items):
    freq = {}
    for item in items:
        freq[item] = freq.get(item, 0) + 1
    return freq

print(count_frequency(["a","b","a","c","b","a"]))
# {'a': 3, 'b': 2, 'c': 1}
```

How Hashing Works Internally

A hash function takes a key and produces a number (the hash code), used to decide which bucket (slot) the value is stored in. When two different keys produce the same bucket, it's called a collision.

Collision Resolution Strategies

- Chaining - each bucket holds a small linked list (or list) of all entries that hash to it.

- Open addressing - on collision, probe for the next available slot (linear probing, quadratic probing, double hashing).
- Load factor - the ratio of stored items to total buckets; hash tables automatically resize (rehash) once the load factor crosses a threshold (commonly ~0.7) to keep operations close to O(1).

> **NOTE:** Because hash tables give near-instant lookups, they are one of the most commonly used data structures in coding interviews - anytime you need to check 'have I seen this before?' quickly, think hash set/dict.

28. Trees, BST, Heaps & Tries

Tree Basics & Terminology

- Root - the topmost node (no parent).
- Leaf - a node with no children.
- Height - the number of edges on the longest path from a node down to a leaf.
- Depth - the number of edges from the root down to a given node.
- Binary Tree - every node has AT MOST two children (left, right).

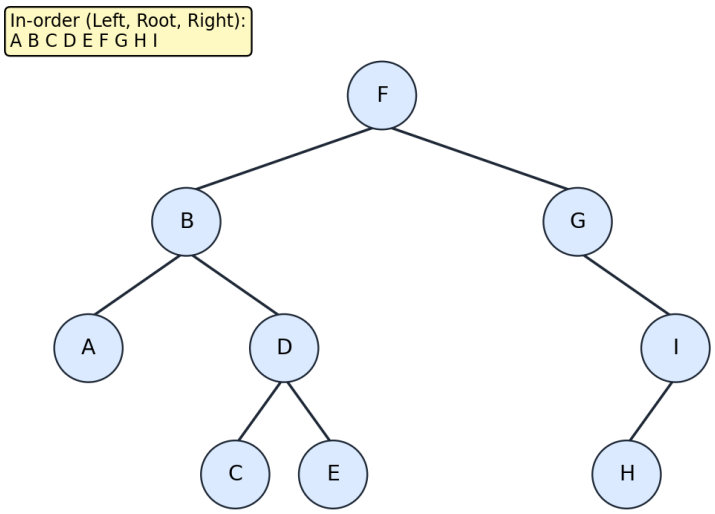

Figure 28.1 — A binary tree

```
class TreeNode:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
```

Tree Traversals — All Four Types

```
def inorder(node, result): # Left, Root, Right -> sorted order in a BST
    if node is None:
        return
    inorder(node.left, result)
    result.append(node.value)
    inorder(node.right, result)

def preorder(node, result): # Root, Left, Right -> useful for copying a tree
    if node is None:
        return
    result.append(node.value)
    preorder(node.left, result)
    preorder(node.right, result)

def postorder(node, result): # Left, Right, Root -> useful for deleting a
tree
    if node is None:
        return
    postorder(node.left, result)
    postorder(node.right, result)
    result.append(node.value)

from collections import deque
def level_order(root): # Breadth-first, level by level - uses a QUEUE
    if root is None:
        return []
    result, queue = [], deque([root])
    while queue:
        node = queue.popleft()
        result.append(node.value)
        if node.left: queue.append(node.left)
        if node.right: queue.append(node.right)
    return result
```

Binary Search Tree (BST)

A BST is a binary tree where, for every node: all values in the left subtree are smaller, and all values in the right subtree are larger. Search, insertion, and deletion run in O(log n) on average for a balanced tree, but degrade to O(n) for a skewed (unbalanced) tree.

```
def insert(node, value):
    if node is None:
        return TreeNode(value)
    if value < node.value:
        node.left = insert(node.left, value)
    else:
        node.right = insert(node.right, value)
    return node

def search(node, value):
    if node is None or node.value == value:
        return node
    if value < node.value:
        return search(node.left, value)
    return search(node.right, value)
```

Balanced Trees (AVL Trees) — Why They Matter

A plain BST can become a 'skewed' tree (essentially a linked list) if you insert sorted data, degrading all operations to O(n). Self-balancing trees like AVL trees and Red-Black trees automatically rotate nodes during insertion/deletion to guarantee O(log n) height at all times.

Heaps (Priority Trees)

A heap is a complete binary tree (filled level by level, left to right) satisfying the heap property: in a min-heap, every parent is smaller than its children; in a max-heap, every parent is larger. Heaps are usually stored as a plain array, not with node pointers.

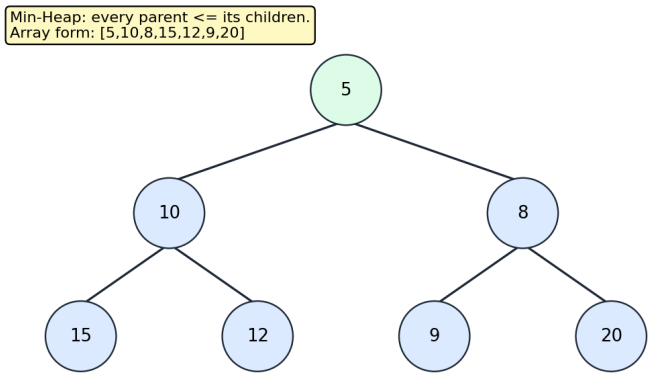

Figure 28.2 — A min-heap and its array representation

- Insert (push) - O(log n): add at the end, then 'bubble up' while smaller than its parent.
- Extract min/max (pop) - O(log n): remove the root, move the last element to root, then 'sink down'.
- Peek min/max - O(1): always the root, arr[0].

- In Python: import heapq, which provides a min-heap on top of a regular list.

Tries (Prefix Trees)

A trie stores strings character by character along tree paths, so words sharing a common prefix share the same initial path. This makes prefix search (autocomplete) extremely fast - O(length of the word), independent of how many words are stored.

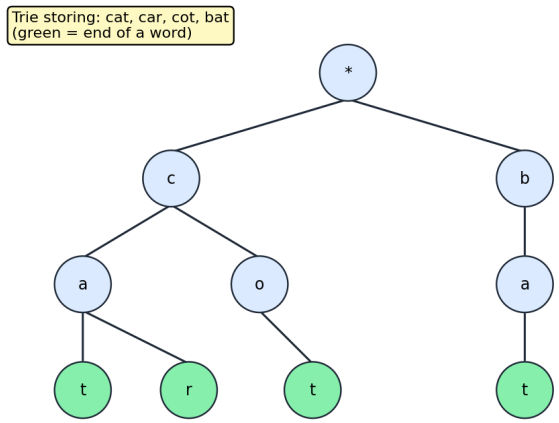

Figure 28.3 — A trie storing cat, car, cot, bat

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end_of_word = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, TrieNode())
        node.is_end_of_word = True

    def search(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                return False
            node = node.children[ch]
        return node.is_end_of_word
```

29. Graphs & Graph Algorithms

A graph is a set of nodes ('vertices') connected by 'edges'. Unlike trees, graphs can have cycles and multiple paths between nodes. Graphs model networks: social connections, maps, the internet.

Representations

```
# Adjacency list - most common, memory-efficient for sparse graphs
graph = {
    "A": ["B", "C"],
    "B": ["A", "D"],
    "C": ["A", "E"],
    "D": ["B", "F"],
    "E": ["C", "F"],
    "F": ["D", "E"],
}

# Adjacency matrix - a 2D grid, good for dense graphs, O(1) edge lookup
```

- Directed graph - edges have a direction (A -> B doesn't imply B -> A).
- Undirected graph - edges go both ways (the example above).
- Weighted graph - each edge has a cost/weight (e.g., road distances).

Breadth-First Search (BFS)

Explores level by level using a Queue. Ideal for finding the shortest path in an unweighted graph.

```
from collections import deque
def bfs(graph, start):
    visited = {start}
    queue = deque([start])
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return order
```

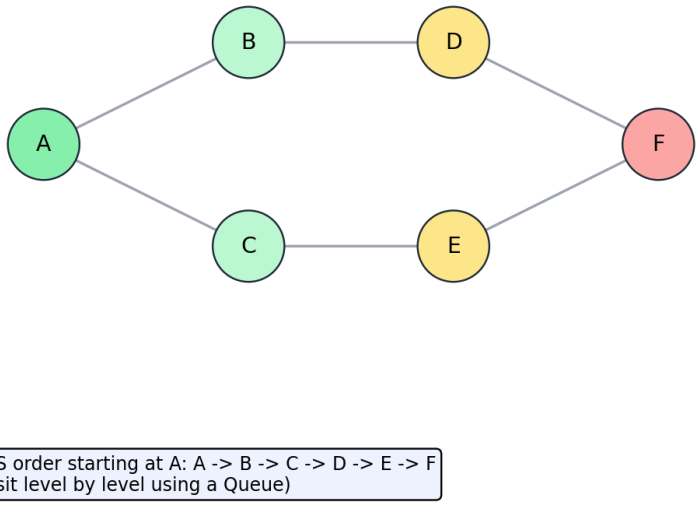

Figure 29.1 — BFS visits nodes level by level using a queue

Depth-First Search (DFS)

Explores as far as possible down one path before backtracking, using a Stack (or recursion).

```
def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()
    visited.add(start)
    print(start, end=" ")
    for neighbor in graph[start]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)

def dfs_iterative(graph, start): # using an explicit stack instead of
recursion
    visited, stack = set(), [start]
    order = []
    while stack:
        node = stack.pop()
        if node not in visited:
            visited.add(node)
            order.append(node)
            stack.extend(reversed(graph[node]))
    return order
```

NOTE: BFS = Queue, explores wide first, finds shortest path in unweighted graphs. DFS = Stack/recursion, explores deep first, useful for cycle detection, topological sort, and maze solving.

Dijkstra's Algorithm (Shortest Path, Weighted Graphs)

Finds the shortest path from a source to all other nodes in a weighted graph (non-negative weights), using a priority queue to always expand the currently-closest unvisited node next.

```
import heapq
def dijkstra(graph, start): # graph[node] = list of (neighbor, weight)
    distances = {node: float("inf") for node in graph}
    distances[start] = 0
    pq = [(0, start)]
    while pq:
        dist, node = heapq.heappop(pq)
        if dist > distances[node]:
            continue
        for neighbor, weight in graph[node]:
            new_dist = dist + weight
            if new_dist < distances[neighbor]:
                distances[neighbor] = new_dist
                heapq.heappush(pq, (new_dist, neighbor))
    return distances
```

Topological Sort

Orders the nodes of a Directed Acyclic Graph (DAG) such that every edge points from an earlier node to a later one - used for task scheduling with dependencies (e.g., course prerequisites).

Union-Find (Disjoint Set Union)

A structure that efficiently tracks which 'group' each element belongs to, supporting near O(1) union (merge two groups) and find (which group is this in?) operations - used in Kruskal's MST algorithm and cycle detection in undirected graphs.

30. Dynamic Programming

Dynamic Programming (DP) solves complex problems by breaking them into overlapping subproblems, solving each subproblem only once, and storing (caching) results to avoid recomputation. DP applies when a problem has optimal substructure (an optimal solution is built from optimal solutions to subproblems) and overlapping subproblems (the same subproblem recurs many times).

Example: Fibonacci — Naive vs Memoized vs Tabulated

```
# Naive recursion - O(2^n), recomputes the same values repeatedly
def fib_naive(n):
    if n <= 1:
        return n
    return fib_naive(n-1) + fib_naive(n-2)

# Top-down DP (memoization) - O(n)
def fib_memo(n, cache={}):
    if n in cache:
        return cache[n]
    if n <= 1:
        return n
    cache[n] = fib_memo(n-1, cache) + fib_memo(n-2, cache)
    return cache[n]

# Bottom-up DP (tabulation) - O(n) time, O(1) space
def fib_tabulation(n):
    if n <= 1:
        return n
    prev2, prev1 = 0, 1
    for _ in range(2, n + 1):
        prev2, prev1 = prev1, prev1 + prev2
    return prev1
```

Classic DP Problems

0/1 Knapsack

```
def knapsack(weights, values, capacity):
    n = len(weights)
    dp = [[0]*(capacity+1) for _ in range(n+1)]
    for i in range(1, n+1):
        for w in range(capacity+1):
            if weights[i-1] <= w:
                dp[i][w] = max(dp[i-1][w], values[i-1] +
dp[i-1][w-weights[i-1]])
            else:
                dp[i][w] = dp[i-1][w]
    return dp[n][capacity]
```

Longest Common Subsequence (LCS)

```
def lcs(a, b):
    m, n = len(a), len(b)
    dp = [[0]*(n+1) for _ in range(m+1)]
    for i in range(1, m+1):
        for j in range(1, n+1):
            if a[i-1] == b[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[m][n]
```

Climbing Stairs (1D DP)

```
def climb_stairs(n): # ways to reach step n, moving 1 or 2 steps at a time
    if n <= 2:
        return n
    dp = [0]*(n+1)
    dp[1], dp[2] = 1, 2
    for i in range(3, n+1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]
```

NOTE: Two DP styles: Top-down (recursion + cache, easier to write first) and Bottom-up (iterative table-filling, often more memory-efficient). Recognize DP problems by phrases like 'count the number of ways', 'find the minimum/maximum', or 'is it possible to reach...'.

31. Greedy Algorithms & Backtracking

Greedy Algorithms

A greedy algorithm makes the locally optimal choice at each step, hoping it leads to a globally optimal solution. Greedy is faster than DP but only works correctly for problems with the 'greedy choice property' - not every problem can be solved greedily.

```
def coin_change_greedy(coins, amount): # works for "nice" coin systems like
[1,5,10,25]
    coins.sort(reverse=True)
    count = 0
    for coin in coins:
        count += amount // coin
        amount %= coin
    return count if amount == 0 else -1
```

- Classic greedy problems: activity selection, Huffman coding, fractional knapsack, Dijkstra's algorithm, Kruskal's/Prim's minimum spanning tree.

Backtracking

Backtracking explores all possible options by building a solution incrementally, abandoning ('backtracking out of') a path as soon as it's known to be invalid, rather than exploring it fully - much faster than pure brute force.

```
def solve_n_queens(n):
    solutions = []
    board = [-1] * n # board[row] = column of the queen in that row

    def is_safe(row, col):
        for r in range(row):
            c = board[r]
            if c == col or abs(c - col) == abs(r - row): # same column or same
diagonal
                return False
        return True

    def backtrack(row):
        if row == n:
            solutions.append(board[:])
            return
        for col in range(n):
            if is_safe(row, col):
                board[row] = col # make a choice
                backtrack(row + 1) # explore further
                board[row] = -1 # undo the choice (backtrack!)

    backtrack(0)
    return solutions
```

- Classic backtracking problems: N-Queens, Sudoku solver, generating all subsets/permutations, word search in a grid, maze solving.

32. Common Problem-Solving Patterns

Most interview/DSA problems are variations of a small number of recurring patterns. Recognizing the pattern tells you which technique and data structure to reach for.

Pattern	When to Use It	Typical Data Structure
Two Pointers	Sorted array, pair/triplet sum problems	Array + 2 indices
Sliding Window	Subarray/substring with a condition	Array + window bounds
Fast & Slow Pointers	Cycle detection, finding the middle	Linked list
BFS / Level Order	Shortest path, level-by-level processing	Queue
DFS / Backtracking	All paths, combinations, permutations	Stack / recursion
Hash Map / Set Lookup	'Have I seen this?', counting, pairs	dict / set
Merge Intervals	Overlapping ranges/intervals	Sorted list + comparisons
Top-K Elements	Kth largest/smallest, frequency ranking	Heap (heapq)

Dynamic Programming	Optimal value, counting ways, overlapping subproblems	Table/array (memo)
Binary Search on Answer	Minimize/maximize a feasible value	Sorted search space

NOTE: When you see a new problem, first ask: is the input sorted (two pointers / binary search)? Do I need all paths (DFS/backtracking)? Do I need the shortest path (BFS)? Am I repeating the same subproblem (DP)? This mapping alone solves a large fraction of interview questions.

PART IV

Practice Roadmap — how to keep growing

33. How to Practice & What to Learn Next

Suggested Study Order

- Week 1-2: Fundamentals - numbers, strings, control flow, lists/tuples/sets/dicts in full depth (Ch. 1-9).
- Week 3: Functions in full depth - all parameter types, lambdas, map/filter/reduce (Ch. 10).
- Week 4: OOP fully - classes, inheritance, magic methods, abstraction (Ch. 11).
- Week 5: Exceptions, files, modules, comprehensions, iterators/generators (Ch. 12-16).
- Week 6: Recursion, decorators/closures, regex (Ch. 17-19) - practice until recursion feels natural.
- Week 7: Big-O + Arrays(+patterns) + Linked Lists + Stacks + Queues (Ch. 20-24).
- Week 8: Searching & Sorting - implement every algorithm from scratch by hand (Ch. 25-26).
- Week 9: Hashing, Trees, Heaps, Tries, Graphs (Ch. 27-29).
- Week 10+: Dynamic Programming, Greedy, Backtracking, then problem patterns (Ch. 30-32), then start solving problems on LeetCode/HackerRank/GeeksforGeeks.

How to Practice Effectively

- Re-implement every algorithm in this guide from memory, without looking, then check your work.
- For every new algorithm, draw it out by hand (boxes and arrows) before coding it.
- Trace through your code line-by-line on paper with a small example, tracking variable values at each step.
- Solve 2-3 problems per topic before moving on; repetition builds pattern recognition (see Chapter 32).
- After solving a problem, study a better solution than yours and understand WHY it's better.

What Comes After This Guide

- Red-Black trees, B-trees, segment trees, Fenwick (Binary Indexed) trees.
- Advanced graph algorithms: A*, Bellman-Ford, Floyd-Warshall, Kruskal's/Prim's MST.
- Advanced DP: Longest Increasing Subsequence, Edit Distance, matrix chain multiplication.
- Concurrency (threading, asyncio) and system design fundamentals once DSA feels solid.

> **TIP:** Consistency beats intensity. 1 focused hour daily for 10 weeks will take you further than occasional 8-hour cram sessions. Use this guide as a living reference - return to any chapter whenever you forget how something works.

End of Guide — Happy Coding!